CS23532 – COMPUTER NETWORKS
EX 12: Echo and Chat using Sockets

AIM :

a) To implement an echo client-server application using TCP and UDP sockets.

b) To implement a chat client-server application using TCP sockets.

ALGORITHM :

a) TCP echo server and client

Server:

1. Create a TCP socket, bind it to the IP address and port number and listen for connections.
2. Accept the connection from the client.
3. Receive the message from the client, display it and send the same message back to the client.
4. Repeat until the client closes the connection, then close the sockets.

Client:

1. Create a TCP socket and connect to the server's IP address and port number.
2. Read a message from the user and send it to the server.
3. Receive the echoed message from the server and display it.
4. Repeat until the user enters exit, then close the socket.

a) UDP echo server and client

Server:

1. Create a UDP socket and bind it to the IP address and port number.
2. Wait for a datagram using recvfrom() and display the message and the client address.
3. Send the same data back to the client using sendto().
4. Stop when the message exit is received.

Client:

1. Create a UDP socket.
2. Read a message from the user and send it to the server using sendto().
3. Receive the echoed message using recvfrom() and display it.
4. Repeat until the user enters exit, then close the socket.

b) TCP chat server and client

Server:

1. Create a TCP socket, bind it to the IP address and port number, listen and accept the client connection.

2. Receive the message from the client and display it.

3. Read the reply from the user and send it to the client.

4. Repeat until either side sends bye, then close the connection.

Client:

1. Create a TCP socket and connect to the server.

2. Read a message from the user and send it to the server.

3. Receive the reply from the server and display it.

4. Repeat until either side sends bye, then close the socket.

PROGRAM :

a) TCP echo server (tcp_echo_server.py)

```
import socket

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(('127.0.0.1', 5000))
server.listen(1)
print("TCP Echo Server is waiting for a connection...")
conn, addr = server.accept()
print("Connected to", addr)
while True:
    data = conn.recv(1024).decode()
    if not data:
        break
    print("Received:", data)
    conn.send(data.encode())
print("Client disconnected")
conn.close()
server.close()
```

a) TCP echo client (tcp_echo_client.py)

```
import socket

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(('127.0.0.1', 5000))
print("Connected to the TCP echo server")
while True:
    msg = input("Enter message (exit to quit): ")
    if msg.lower() == 'exit':
```

```
        break
    client.send(msg.encode())
    reply = client.recv(1024).decode()
    print("Echo from server:", reply)
client.close()
```

a) UDP echo server (udp_echo_server.py)

```
import socket

server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind(('127.0.0.1', 6000))
print("UDP Echo Server is ready...")
while True:
    data, addr = server.recvfrom(1024)
    msg = data.decode()
    if msg.lower() == 'exit':
        print("Server stopped")
        break
    print("Received from", addr, ":", msg)
    server.sendto(data, addr)
server.close()
```

a) UDP echo client (udp_echo_client.py)

```
import socket

client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server_addr = ('127.0.0.1', 6000)
while True:
    msg = input("Enter message (exit to quit): ")
    client.sendto(msg.encode(), server_addr)
    if msg.lower() == 'exit':
        break
    data, addr = client.recvfrom(1024)
    print("Echo from server:", data.decode())
client.close()
```

b) Chat server (chat_server.py)

```
import socket

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(('127.0.0.1', 5001))
server.listen(1)
print("Chat server is waiting for a client...")
conn, addr = server.accept()
print("Client connected from", addr)
while True:
    data = conn.recv(1024).decode()
    if not data or data.lower() == 'bye':
        print("Client left the chat")
        break
```

```
    print("Client:", data)
    msg = input("Server: ")
    conn.send(msg.encode())
    if msg.lower() == 'bye':
        break
conn.close()
server.close()
```

b) Chat client (chat_client.py)

```
import socket

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(('127.0.0.1', 5001))
print("Connected to the chat server")
while True:
    msg = input("Client: ")
    client.send(msg.encode())
    if msg.lower() == 'bye':
        break
    data = client.recv(1024).decode()
    if not data or data.lower() == 'bye':
        print("Server left the chat")
        break
    print("Server:", data)
client.close()
```

INPUT AND OUTPUT :

a) TCP echo server and client

Input: Hello Server, Welcome to Computer Networks, exit

Server output

```
TCP Echo Server is waiting for a connection...
Connected to ('127.0.0.1', 42262)
Received: Hello Server
Received: Welcome to Computer Networks
Client disconnected
```

Client output

```
Connected to the TCP echo server
Enter message (exit to quit): Hello Server
Echo from server: Hello Server
Enter message (exit to quit): Welcome to Computer Networks
Echo from server: Welcome to Computer Networks
Enter message (exit to quit): exit
```

a) UDP echo server and client

Input: Hello UDP Server, Transport layer is connectionless, exit

Server output

```
UDP Echo Server is ready...
```

```
Received from ('127.0.0.1', 38542) : Hello UDP Server
Received from ('127.0.0.1', 38542) : Transport layer is connectionless
Server stopped
```

Client output

```
Enter message (exit to quit): Hello UDP Server
Echo from server: Hello UDP Server
Enter message (exit to quit): Transport layer is connectionless
Echo from server: Transport layer is connectionless
Enter message (exit to quit): exit
```

b) Chat server and client

Input: Client: Hello Server, Please explain the TCP three-way handshake, Thank you, bye. Server: Hello Client, how can I help you?, SYN, SYN-ACK and ACK are exchanged, You are welcome

Server output

```
Chat server is waiting for a client...
Client connected from ('127.0.0.1', 34002)
Client: Hello Server
Server: Hello Client, how can I help you?
Client: Please explain the TCP three-way handshake
Server: SYN, SYN-ACK and ACK are exchanged
Client: Thank you
Server: You are welcome
Client left the chat
```

Client output

```
Connected to the chat server
Client: Hello Server
Server: Hello Client, how can I help you?
Client: Please explain the TCP three-way handshake
Server: SYN, SYN-ACK and ACK are exchanged
Client: Thank you
Server: You are welcome
Client: bye
```

RESULT:

Thus, the echo client-server application using TCP and UDP sockets and the chat client-server application using TCP sockets were implemented and executed successfully.